

Building a Feedforward Neural Network from Scratch in Python

April 10th 2019 ★ 31,560 reads

6    

```
31 def __init__(self, path):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, 'requests.log'),
39                          'a')
40         self.file.seek(0)
41         self.fingerprints.update(e.request for e in self.requests)
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool('SUPERFUTUR_JUNKIE')
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```



@NKumar

NiranjanKumar

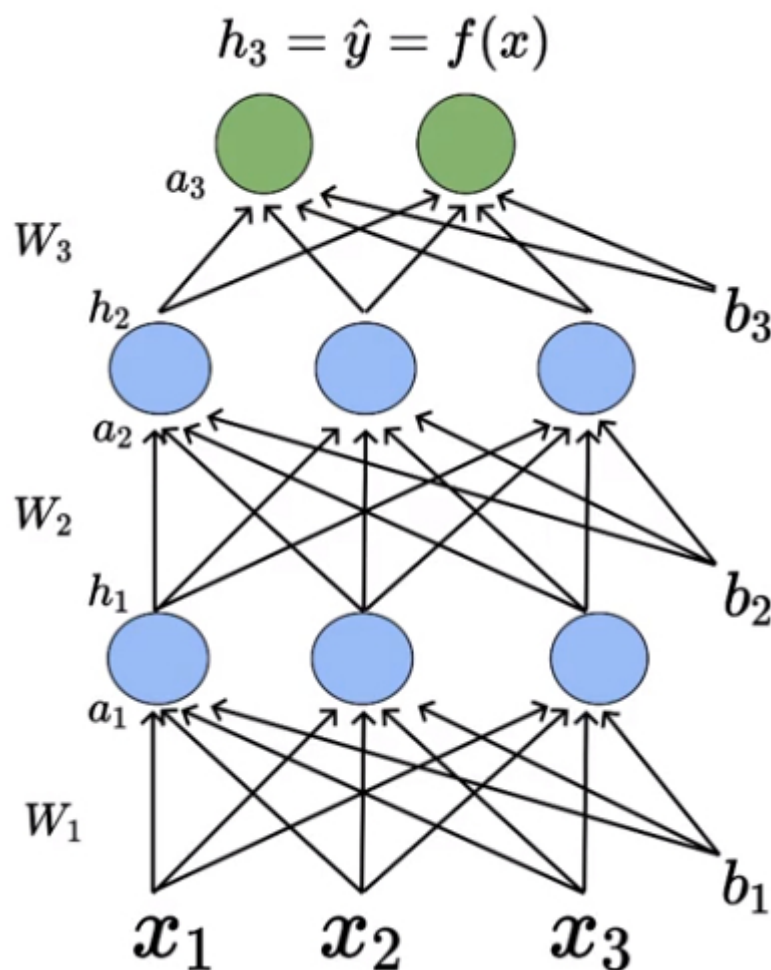
DeepLearning Enthusiast. Data Science Writer @marktechpost.com

[in](#) [tw](#) [gh](#)

In this post, we will see how to implement the feedforward neural network from scratch in python. This is a follow up to my previous post on the feedforward neural networks.

Feedforward Neural Networks

Feedforward neural networks are also known as **Multi-layered Network of Neurons** (MLN). These network of models are called feedforward because the information only travels forward in the neural network, through the input nodes then through the hidden layers (single or many layers) and finally through the output nodes.



Traditional models such as McCulloch Pitts, Perceptron and Sigmoid neuron models capacity is limited to linear functions. To handle the complex non-linear decision boundary between input and the output we are using the Multi-layered Network of Neurons.

To understand the feedforward neural network learning algorithm and the computations present in the network, kindly refer to my previous post on Feedforward Neural Networks.

Deep Learning: Feedforward Neural Networks Explained

Coding Part



In the coding section, we will be covering the following topics.

1. **Generate data that is not linearly separable**
2. **Train with Sigmoid Neuron and see performance**
3. **Write from scratch our first feedforward network**
4. **Train the FF network on the data and compare with Sigmoid Neuron**
5. **Write a generic class for a FF network**
6. **Train generic class on binary classification**
7. **Train a FF network for multi-class data using a cross-entropy loss function**

If you want to skip the theory part and get into the code right away,

Niranjankumar-c/Feedforward_NeuralNetworks

*PS: If you are interested in converting the code into **R**, send me a message once it is done. I will feature your work here and also on the GitHub page.*

Before we start building our network, first we need to import the required libraries. We are importing the `numpy` to evaluate the matrix multiplication and dot product between two vectors, `matplotlib` to visualize the data and from `thesklearn` package we are importing functions to generate data and evaluate the network performance.

```
import numpy as np
import matplotlib.pyplot as plt
```



```
import matplotlib.pyplot as plt
import matplotlib.colors
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error
from tqdm import tqdm_notebook

from sklearn.preprocessing import OneHotEncoder
from sklearn.datasets import make_blobs
```

Generate Dummy Data

Remember that we are using feedforward neural networks because we wanted to deal with non-linearly separable data. In this section, we will see how to randomly generate non-linearly separable data.

```
#creating my own color map for better visualization
my_cmap = matplotlib.colors.LinearSegmentedColormap.from_list("", ["red","yellow","green"])

#Generating 1000 observations with 4 labels - multi class
data, labels = make_blobs(n_samples=1000, centers=4, n_features=2, random_state=0)
print(data.shape, labels.shape)

#visualize the data
plt.scatter(data[:,0], data[:,1], c=labels, cmap=my_cmap)
plt.show()

#converting the multi-class to binary
labels_orig = labels
labels = np.mod(labels_orig, 2)
plt.scatter(data[:,0], data[:,1], c=labels, cmap=my_cmap)
plt.show()

#split the binary data
X_train, X_val, Y_train, Y_val = train_test_split(data, labels, stratify=labels,
random_state=0)
print(X_train.shape, X_val.shape)
```

To generate data randomly we will use `make_blobs` to generate blobs of points with a Gaussian distribution. I have generated 1000 data points in 2D space with four blobs `centers=4` as a multi-class classification prediction problem. Each data point has two inputs and 0, 1, 2 or 3 class labels. The code present in **Line 9, 10** helps to visualize the data using a scatter plot. We can see that they are 4 centers present and the data is linearly separable (almost).



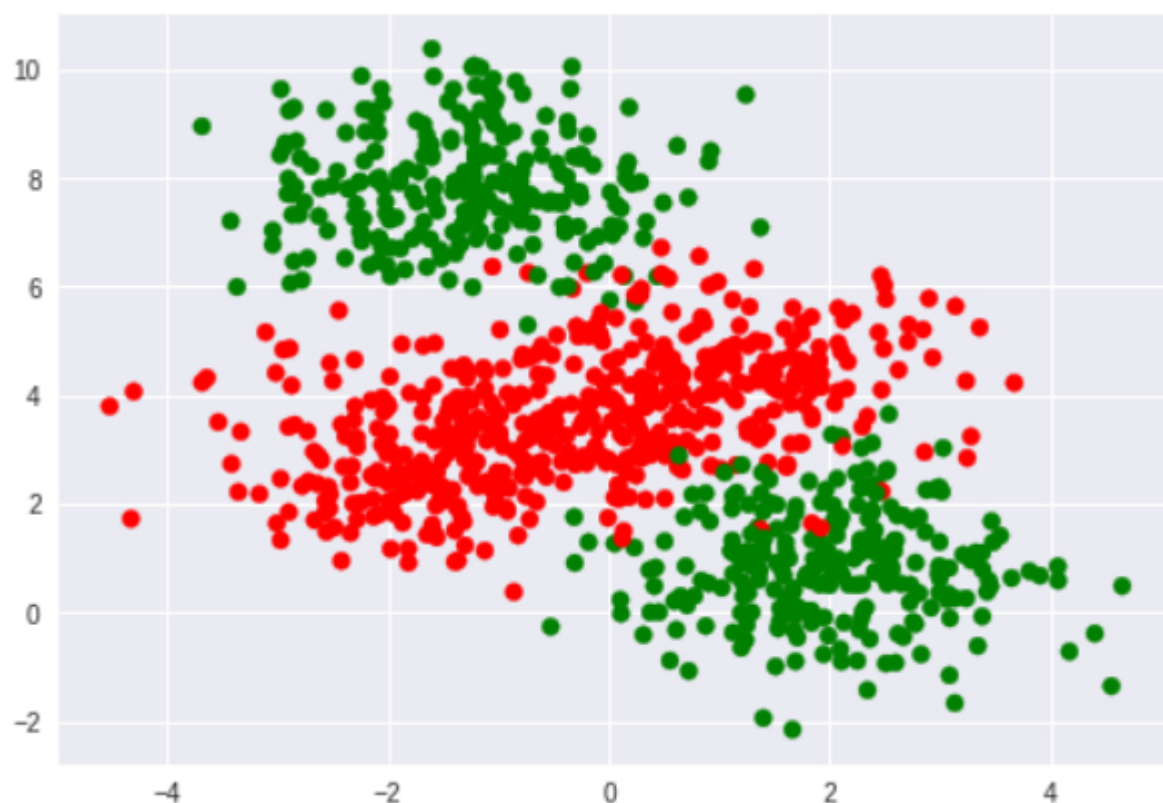
Multi-Class Data

In the above plot, I was able to represent 3 Dimensions—2 Inputs and class labels as colors using a simple scatter plot. Note that `make_blobs()` function will generate linearly separable data, but we need to have non-linearly separable data for binary classification.

```
labels_orig = labels
```

```
labels = np.mod(labels_orig, 2)
```

One way to convert the 4 classes to binary classification is to take the remainder of these 4 classes when they are divided by 2 so that I can get the new labels as 0 and 1.



Binary Class Data

From the plot, we can see that the centers of blobs are merged such that we now have a binary classification problem where the decision boundary is not linear. Once we have our data ready, I have used the `train_test_split` function to split the data for training and validation in the ratio of 90:10

Train with Sigmoid Neuron

Before we start training the data on the sigmoid neuron. We will build our model inside a class called

Before we start training our data on the sigmoid neuron, we will build our model into a class called *SigmoidNeuron*.

```
class SigmoidNeuron:
    #intialization
    def __init__(self):
        self.w = None
        self.b = None
    #forward pass
    def perceptron(self, x):
        return np.dot(x, self.w.T) + self.b

    def sigmoid(self, x):
        return 1.0/(1.0 + np.exp(-x))
    #updating the gradients using mean squared error loss
    def grad_w_mse(self, x, y):
        y_pred = self.sigmoid(self.perceptron(x))
        return (y_pred - y) * y_pred * (1 - y_pred) * x

    def grad_b_mse(self, x, y):
        y_pred = self.sigmoid(self.perceptron(x))
        return (y_pred - y) * y_pred * (1 - y_pred)
    #updating the gradients using cross entropy loss
    def grad_w_ce(self, x, y):
        y_pred = self.sigmoid(self.perceptron(x))
        if y == 0:
            return y_pred * x
        elif y == 1:
            return -1 * (1 - y_pred) * x
        else:
            raise ValueError("y should be 0 or 1")

    def grad_b_ce(self, x, y):
        y_pred = self.sigmoid(self.perceptron(x))
        if y == 0:
            return y_pred
        elif y == 1:
            return -1 * (1 - y_pred)
        else:
            raise ValueError("y should be 0 or 1")
    #model fit method
    def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, loss_fn="mse",
display_loss=False):
```



```

# initialise w, b
if initialise:
    self.w = np.random.randn(1, X.shape[1])
    self.b = 0

if display_loss:
    loss = {}

for i in tqdm_notebook(range(epochs), total=epochs, unit="epoch"):
    dw = 0
    db = 0
    for x, y in zip(X, Y):
        if loss_fn == "mse":
            dw += self.grad_w_mse(x, y)
            db += self.grad_b_mse(x, y)
        elif loss_fn == "ce":
            dw += self.grad_w_ce(x, y)
            db += self.grad_b_ce(x, y)

    m = X.shape[1]
    self.w -= learning_rate * dw/m
    self.b -= learning_rate * db/m

    if display_loss:
        Y_pred = self.sigmoid(self.perceptron(X))
        if loss_fn == "mse":
            loss[i] = mean_squared_error(Y, Y_pred)
        elif loss_fn == "ce":
            loss[i] = log_loss(Y, Y_pred)

    if display_loss:
        plt.plot(loss.values())
        plt.xlabel('Epochs')
        if loss_fn == "mse":
            plt.ylabel('Mean Squared Error')
        elif loss_fn == "ce":
            plt.ylabel('Log Loss')
        plt.show()

def predict(self, X):
    Y_pred = []
    for x in X:
        y_pred = self.sigmoid(self.perceptron(x))
        Y_pred.append(y_pred)
    return np.array(Y_pred)

```

In the class `SigmoidNeuron` we have 9 functions, I will walk you through these functions one by one and explain what they are doing.

```
def __init__(self):  
    self.w = None  
    self.b = None
```

The `__init__` function (constructor function) helps to initialize the parameters of sigmoid neuron **w** weights and **b** biases to None.

```
#forward pass  
def perceptron(self, x):  
    return np.dot(x, self.w.T) + self.b  
def sigmoid(self, x):  
    return 1.0/(1.0 + np.exp(-x))
```

Next, we will define two functions `perceptron` and `sigmoid` which characterizes the forward pass. In case of a sigmoid neuron forward pass involves two steps

- `perceptron` — Computes the dot product between the input **x** & weights **w** and adds bias **b**
- `sigmoid` — Takes the output of `perceptron` and applies the sigmoid (logistic) function on top of it.

```
#updating the gradients using mean squared error loss
def grad_w_mse(self, x, y):
    .....
def grad_b_mse(self, x, y):
    .....
#updating the gradients using cross entropy loss
def grad_w_ce(self, x, y):
    .....
def grad_b_ce(self, x, y):
    .....
```

The next four functions characterize the gradient computation. I have written two separate functions for updating weights **w** and biases **b** using mean squared error loss and cross-entropy loss.

```
def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, loss_fn="mse",
display_loss=False):
    .....
    return
```

Next, we define ‘fit’ method that accepts a few parameters,

x —Inputs

y —Labels

`epochs` —Number of epochs we will allow our algorithm through iterate on the data, default value set to 1

`learning_rate` —The magnitude of change for our weights during each step through our training data, default value set to 1

`initialise` —To randomly initialize the parameters of the model or not. If it is set to True weights will be initialized, you can set it to False if you want to retrain the trained model.

`loss_fn` —To select the loss function for the algorithm to update the parameters. It can be “mse” or “ce”

`display_loss` —Boolean Variable indicating whether to show the decrease of loss for each epoch

In the `fit` method, we go through the data passed through parameters X and Y and compute the update values for the parameters either using mean squared loss or cross entropy loss. Once we the update value we go and update the weights and bias terms (*Line 49–62*).

```
def predict(self, X):
```

Now we define our predict function takes inputs `x` as an argument, which it expects to be an `numpy` array. In the predict function, we will compute the forward pass of each input with the trained model

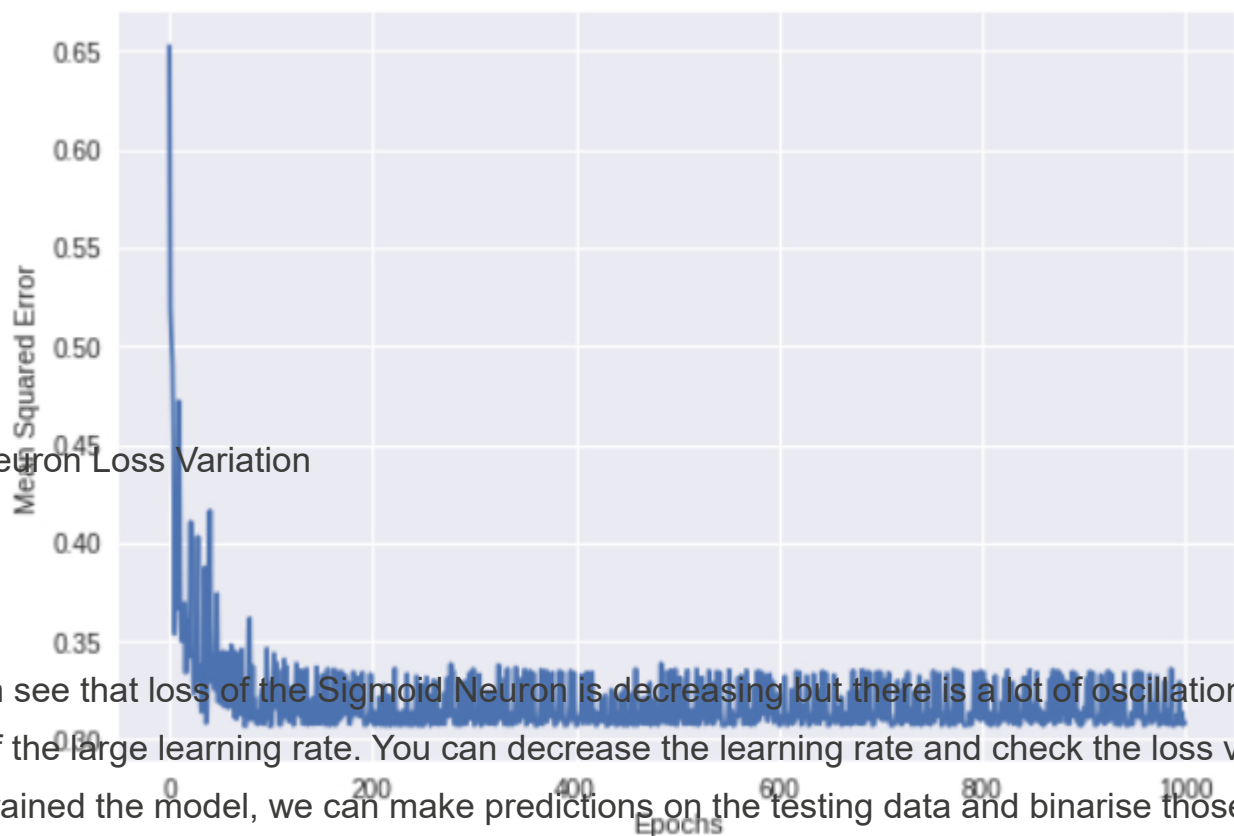
and send back a numpy array which contains the predicted value of each input data.

```
#create a class object
sn = SigmoidNeuron()

#train the model
sn.fit(X_train, Y_train, epochs=1000, learning_rate=0.5, display_loss=True)
#prediction on training data
Y_pred_train = sn.predict(X_train)
Y_pred_binarised_train = (Y_pred_train >= 0.5).astype("int").ravel()
#prediction on testing data
Y_pred_val = sn.predict(X_val)
Y_pred_binarised_val = (Y_pred_val >= 0.5).astype("int").ravel()
#model accuracy
accuracy_train = accuracy_score(Y_pred_binarised_train, Y_train)
accuracy_val = accuracy_score(Y_pred_binarised_val, Y_val)
print("Training accuracy", round(accuracy_train, 2))
print("Validation accuracy", round(accuracy_val, 2))

#visualizing the results
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```

Now we will train our data on the sigmoid neuron which we created. First, we instantiate the Sigmoid Neuron Class and then call the `fit` method on the training data with 1000 epochs and learning rate set to 1 (These values are arbitrary not the optimal values for this data, you can play around these values and find the best number of epochs and the learning rate). By default, the loss function is set to *mean square error loss* but you can change it to *cross entropy loss* as well.

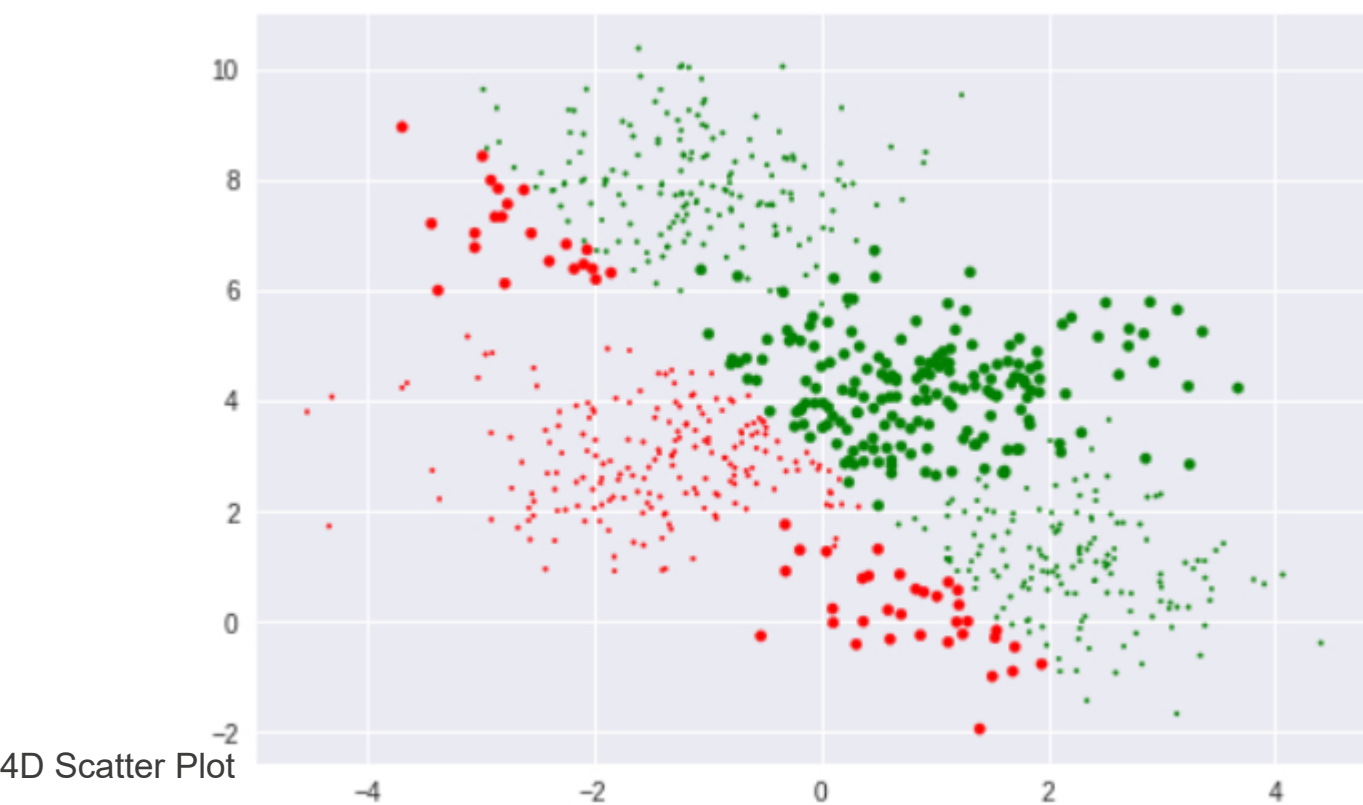


Sigmoid Neuron Loss Variation

As you can see that loss of the Sigmoid Neuron is decreasing but there is a lot of oscillations may be because of the large learning rate. You can decrease the learning rate and check the loss variation. Once we trained the model, we can make predictions on the testing data and binarise those predictions by taking 0.5 as the threshold. We can compute the training and validation accuracy of the model to evaluate the performance of the model and check for any scope of improvement by changing the number of epochs or learning rate.

```
#visualizing the results
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```

To know which of the data points that the model is predicting correctly or not for each point in the training set. we will use the scatter plot function from `matplotlib.pyplot`. The function takes two inputs as the first and second features, for the color I have used `Y_pred_binarised_train` and defined a custom 'cmap' for visualization. As you can see that the size of each point is different in the below plot.



```
s=15*(np.abs(Y_pred_binarised_train-Y_train)+.2)
```

The formula takes the absolute difference between the predicted value and the actual value.

- If the ground truth is equal to the predicted value then size = 3
- If the ground truth is not equal to the predicted value the size = 18

All the small points in the plot indicate that the model is predicting those observations correctly and

large points indicate that those observations are incorrectly classified.



4D Scatter Plot

In this plot, we are able to represent 4 Dimensions—Two input features, color to indicate different labels and size of the point indicates whether it is predicted correctly or not. The important note from the plot is that sigmoid neuron is not able to handle the non-linearly separable data.

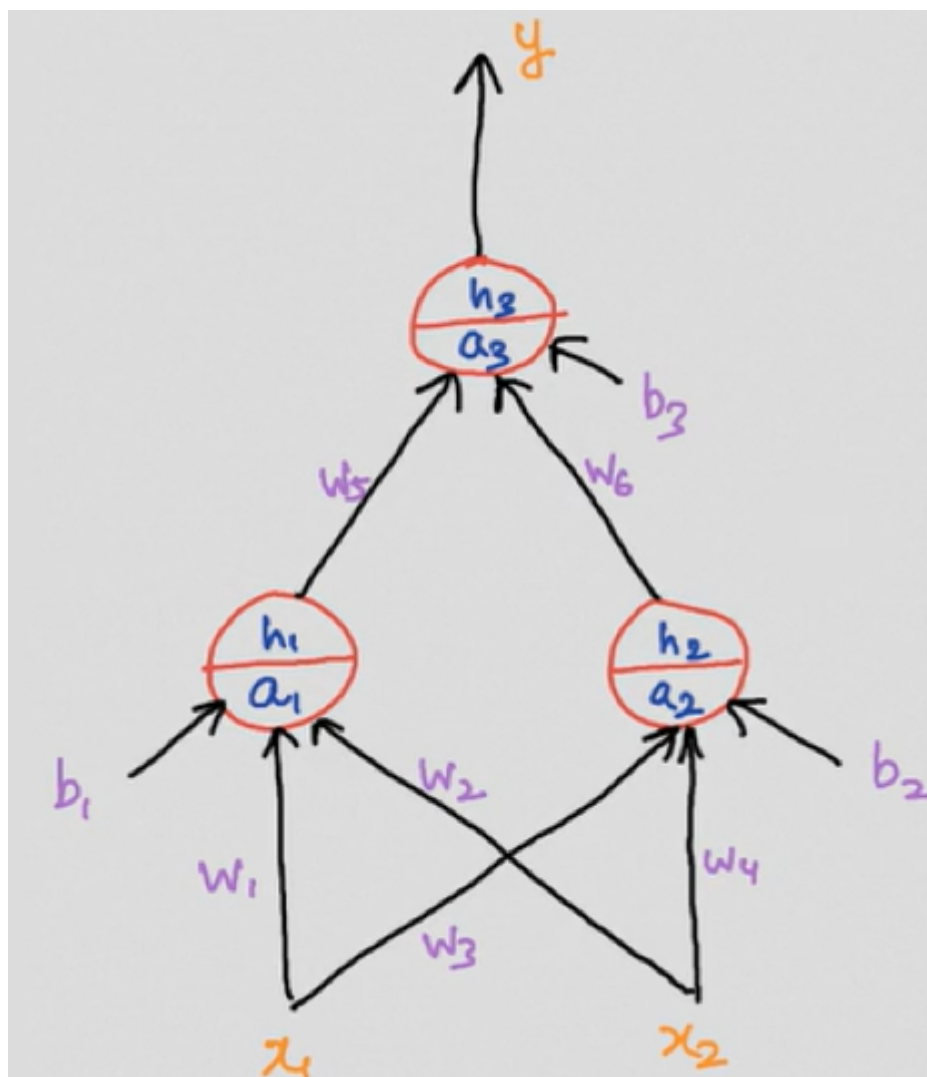
If you want to learn sigmoid neuron learning algorithm in detail with math check out my previous post.

Sigmoid Neuron Learning Algorithm Explained With Math

Write First Feedforward Neural Network

Write First Feedforward Neural Network

In this section, we will take a very simple feedforward neural network and build it from scratch in python. The network has three neurons in total—two in the first hidden layer and one in the output layer. For each of these neurons, pre-activation is represented by 'a' and post-activation is represented by 'h'. In the network, we have a total of 9 parameters—6 weight parameters and 3 bias terms.



Simple Feedforward Network

Similar to the Sigmoid Neuron implementation, we will write our neural network in a class called *FirstFFNetwork*.

```
class FirstFFNetwork:
```

```
    #intialize the parameters
```

```
    def __init__(self):
```

```
        self.w1 = np.random.randn()
```

```
        self.w2 = np.random.randn()
```

```
        self.w3 = np.random.randn()
```

```
        self.w4 = np.random.randn()
```

```
        self.w5 = np.random.randn()
```

```
        self.w6 = np.random.randn()
```

```
        self.b1 = 0
```

```
        self.b2 = 0
```

```
        self.b3 = 0
```

```
    def sigmoid(self, x):
```

```
        return 1.0/(1.0 + np.exp(-x))
```

```
    def forward_pass(self, x):
```

```
        #forward pass - preactivation and activation
```

```
        self.x1, self.x2 = x
```

```
        self.a1 = self.w1*self.x1 + self.w2*self.x2 + self.b1
```

```
        self.h1 = self.sigmoid(self.a1)
```

```
        self.a2 = self.w3*self.x1 + self.w4*self.x2 + self.b2
```

```
        self.h2 = self.sigmoid(self.a2)
```

```
        self.a3 = self.w5*self.h1 + self.w6*self.h2 + self.b3
```

```
        self.h3 = self.sigmoid(self.a3)
```

```
        return self.h3
```

```
    def grad(self, x, y):
```

```
        #back propagation
```

```
        self.forward_pass(x)
```

```
        self.dw5 = (self.h3-y) * self.h3*(1-self.h3) * self.h1
```

```
        self.dw6 = (self.h3-y) * self.h3*(1-self.h3) * self.h2
```

```
        self.db3 = (self.h3-y) * self.h3*(1-self.h3)
```

```
        self.dw1 = (self.h3-y) * self.h3*(1-self.h3) * self.w5 * self.h1*(1-self.h1) * self.x1
```

```
        self.dw2 = (self.h3-y) * self.h3*(1-self.h3) * self.w5 * self.h1*(1-self.h1) * self.x2
```

```
        self.db1 = (self.h3-y) * self.h3*(1-self.h3) * self.w5 * self.h1*(1-self.h1)
```

```
        self.dw3 = (self.h3-y) * self.h3*(1-self.h3) * self.w6 * self.h2*(1-self.h2) * self.x1
```

```
        self.dw4 = (self.h3-y) * self.h3*(1-self.h3) * self.w6 * self.h2*(1-self.h2) * self.x2
```

```
        self.db2 = (self.h3-y) * self.h3*(1-self.h3) * self.w6 * self.h2*(1-self.h2)
```

```
    def fit(self, X, Y, epochs=1, learning_rate=1, initialize=True, display_loss=False):
```

```
def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, display_loss=False):
```

```
    # initialise w, b
```

```
    if initialise:
```

```
        self.w1 = np.random.randn()
```

```
        self.w2 = np.random.randn()
```

```
        self.w3 = np.random.randn()
```

```
        self.w4 = np.random.randn()
```

```
        self.w5 = np.random.randn()
```

```
        self.w6 = np.random.randn()
```

```
        self.b1 = 0
```

```
        self.b2 = 0
```

```
        self.b3 = 0
```

```
    if display_loss:
```

```
        loss = {}
```

```
    for i in tqdm_notebook(range(epochs), total=epochs, unit="epoch"):
```

```
        dw1, dw2, dw3, dw4, dw5, dw6, db1, db2, db3 = [0]*9
```

```
        for x, y in zip(X, Y):
```

```
            self.grad(x, y)
```

```
            dw1 += self.dw1
```

```
            dw2 += self.dw2
```

```
            dw3 += self.dw3
```

```
            dw4 += self.dw4
```

```
            dw5 += self.dw5
```

```
            dw6 += self.dw6
```

```
            db1 += self.db1
```

```
            db2 += self.db2
```

```
            db3 += self.db3
```

```
    m = X.shape[1]
```

```
    self.w1 -= learning_rate * dw1 / m
```

```
    self.w2 -= learning_rate * dw2 / m
```

```
    self.w3 -= learning_rate * dw3 / m
```

```
    self.w4 -= learning_rate * dw4 / m
```

```
    self.w5 -= learning_rate * dw5 / m
```

```
    self.w6 -= learning_rate * dw6 / m
```

```
    self.b1 -= learning_rate * db1 / m
```

```
    self.b2 -= learning_rate * db2 / m
```

```
    self.b3 -= learning_rate * db3 / m
```

```
    if display_loss:
```

```
        Y_pred = self.predict(X)
```

```
        loss[i] = mean_squared_error(Y_pred, Y)
```

```
    if display_loss:
```

```
        plt.plot(loss.values())
```

```
        plt.xlabel('Epochs')
```

```

plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.show()

def predict(self, X):

    #predicting the results on unseen data
    Y_pred = []
    for x in X:
        y_pred = self.forward_pass(x)
        Y_pred.append(y_pred)
    return np.array(Y_pred)

```

In the class `FirstFFNetwork` we have 6 functions, we will go over these functions one by one.

```

def __init__(self):
    .....

```

The `__init__` function initializes all the parameters of the network including weights and biases. Unlike the sigmoid neuron where we have only two parameters in the neural network, we have 9 parameters to be initialized. All the 6 weights are initialized randomly and 3 biases are set to zero.

```

def sigmoid(self, x):
    return 1.0/(1.0 + np.exp(-x))

```

Next, we define the sigmoid function used for post-activation for each of the neurons in the network.


```
def forward_pass(self, x):  
  
    #forward pass - preactivation and activation  
    self.x1, self.x2 = x  
    self.a1 = self.w1*self.x1 + self.w2*self.x2 + self.b1  
    self.h1 = self.sigmoid(self.a1)  
    self.a2 = self.w3*self.x1 + self.w4*self.x2 + self.b2  
    self.h2 = self.sigmoid(self.a2)  
    self.a3 = self.w5*self.h1 + self.w6*self.h2 + self.b3  
    self.h3 = self.sigmoid(self.a3)  
    return self.h3
```

Now we have the forward pass function, which takes an input $\mathbf{x}$ and computes the output. First, I have initialized two local variables and equated to input $\mathbf{x}$ which has 2 features.

For each of these 3 neurons, two things will happen,

Pre-activation represented by 'a': It is a weighted sum of inputs plus the bias.

Activation represented by 'h': Activation function is Sigmoid function.

The pre-activation for the first neuron is given by,

$$a_1 = w_1 * x_1 + w_2 * x_2 + b_1$$

To get the post-activation value for the first neuron we simply apply the logistic function to the output of pre-activation a_1 .

$$h_1 = \text{sigmoid}(a_1)$$

Repeat the same process for the second neuron to get a_2 and h_2 .

The outputs of the two neurons present in the first hidden layer will act as the input to the third neuron. The pre-activation for the third neuron is given by,

$$a_3 = w_5 * h_1 + w_6 * h_2 + b_3$$

and applying the sigmoid on a_3 will give the final predicted output.

```
def grad(self, x, y):  
    #back propagation  
    .....
```

Next, we have the `grad` function which takes inputs **x** and **y** as arguments and computes the forward pass. Based on the forward pass it computes the partial derivatives of these weights with respect to the loss function, which is mean squared error loss in this case.

Note: In this post, I am not explaining how do we arrive at these partial derivatives for the parameters. Just consider this function as a black box for now, in my next article I will explain how do we compute these partial derivatives in backpropagation.

```
def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, display_loss=False):  
    .....
```

Then, we have the `fit` function similar to the sigmoid neuron. In this function, we iterate through each data point, compute the partial derivatives by calling the `grad` function and store those values in a new variable for each parameter (*Line 63–75*). Then, we go ahead and update the values of all the parameters (*Line 77–87*). We also have the `display_loss` condition, if set to `True` it will display the plot of network loss variation across all the epochs.

```
def predict(self, X):  
    #predicting the results on unseen data  
    .....
```

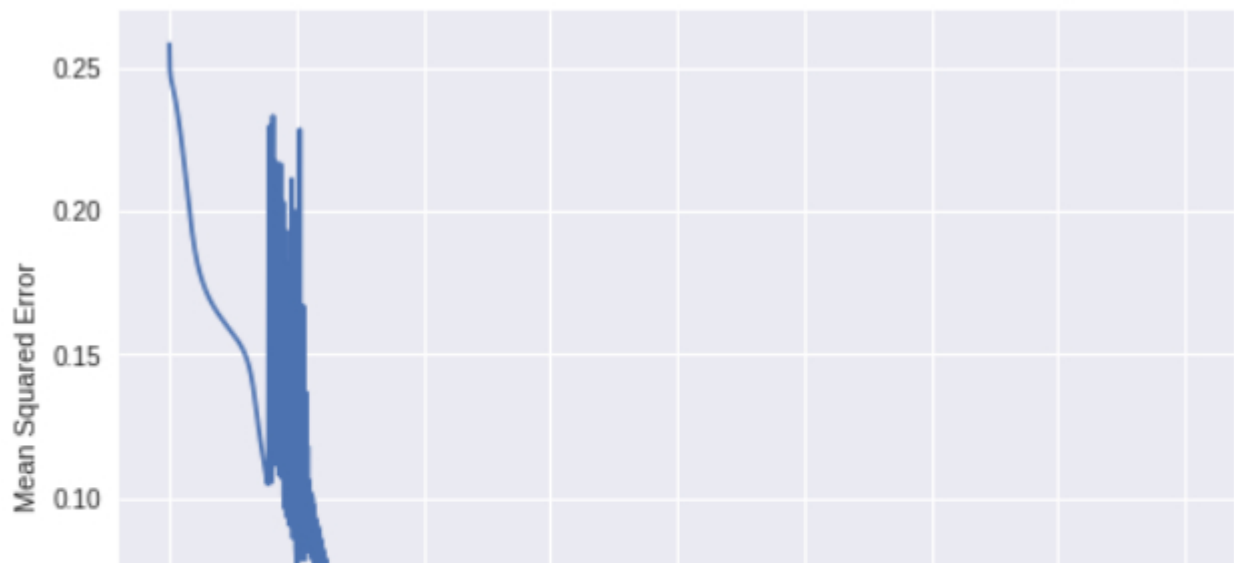
Finally, we have the predict function that takes a large set of values as inputs and compute the predicted value for each input by calling the forward_pass function on each of the input.

Train the FF network on the data

We will now train our data on the Feedforward network which we created. First, we instantiate the FirstFFNetwork Class and then call the fit method on the training data with 2000 epochs and learning rate set to 0.01.

```
ffn = FirstFFNetwork()
#train the model on the data
ffn.fit(X_train, Y_train, epochs=2000, learning_rate=.01, display_loss=True)
#predictions
Y_pred_train = ffn.predict(X_train)
Y_pred_binarised_train = (Y_pred_train >= 0.5).astype("int").ravel()
Y_pred_val = ffn.predict(X_val)
Y_pred_binarised_val = (Y_pred_val >= 0.5).astype("int").ravel()
accuracy_train = accuracy_score(Y_pred_binarised_train, Y_train)
accuracy_val = accuracy_score(Y_pred_binarised_val, Y_val)
#model performance
print("Training accuracy", round(accuracy_train, 2))
print("Validation accuracy", round(accuracy_val, 2))

#visualize the predictions
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```



```
#visualize the predictions
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```

To get a better idea about the performance of the neural network, we will use the same 4D visualization plot that we used in sigmoid neuron and compare it with the sigmoid neuron model.



As you can see most of the points are classified correctly by the neural network. The key takeaway is that just by combining three sigmoid neurons we are able to solve the problem of non-linearly separable data.

Generic Class for Feedforward Neural Network

In this section, we will write a generic class where it can generate a neural network, by taking the number of hidden layers and the number of neurons in each hidden layer as input parameters. The generic class also takes the number of inputs as parameter earlier we have only two inputs but now we can have 'n' dimensional inputs as well.

Note: In this case, I am considering the network for binary classification only.



Search...



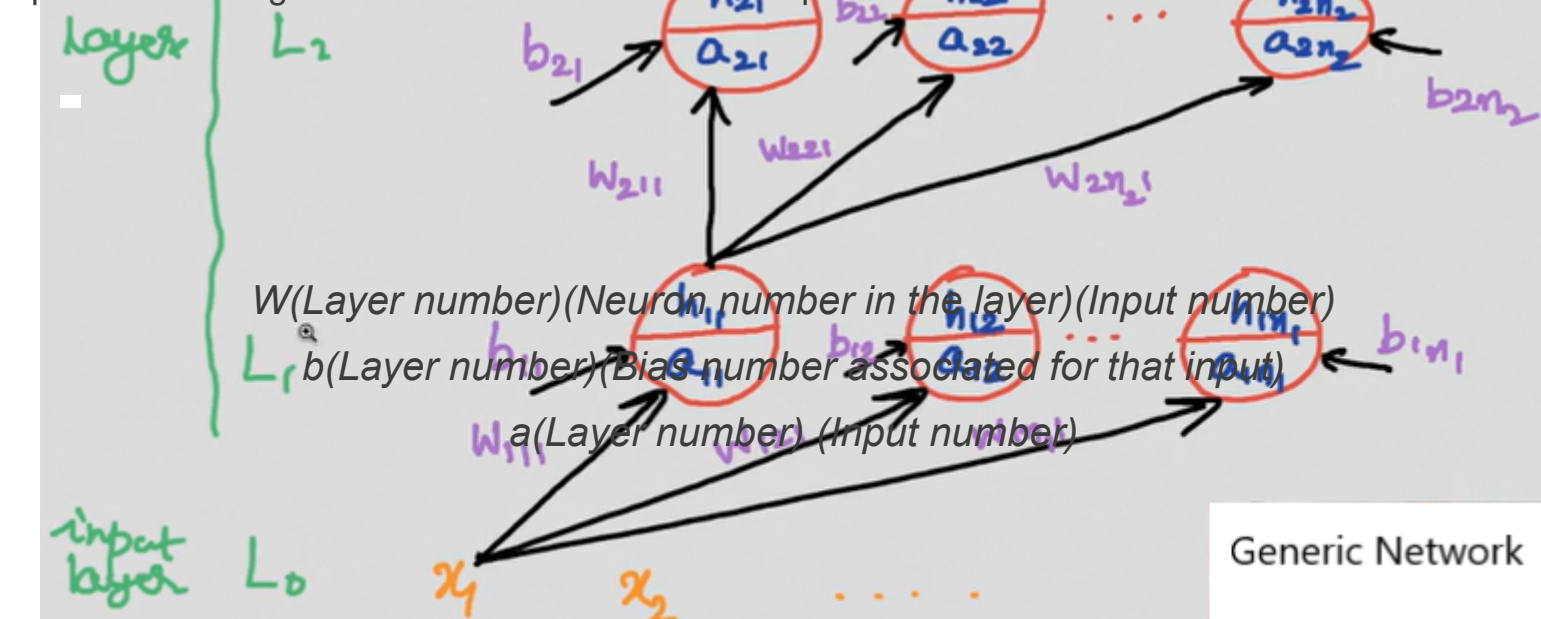
bybit 400,000 USDT To Be Won!



Building a Feedforward Neural Network from Scratch in Python by @NKumar

Generic Feedforward Network

Before we start to write code for the generic neural network, let us understand the format of indices to represent the weights and biases associated with a particular neuron.



W_{111} —Weight associated with the first neuron present in the first hidden layer connected to the first input.

W_{112} —Weight associated with the first neuron present in the first hidden layer connected to the second input.

b_{11} —Bias associated with the first neuron present in the first hidden layer.

b_{12} —Bias associated with the second neuron present in the first hidden layer.

The Code:

```
class FFSNNetwork:

    def __init__(self, n_inputs, hidden_sizes=[2]):
        #intialize the inputs
        self.nx = n_inputs
        self.ny = 1
        self.nh = len(hidden_sizes)
        self.sizes = [self.nx] + hidden_sizes + [self.ny]

        self.W = {}
        self.B = {}
        for i in range(self.nh+1):
            self.W[i+1] = np.random.randn(self.sizes[i], self.sizes[i+1])
            self.B[i+1] = np.zeros((1, self.sizes[i+1]))

    def sigmoid(self, x):
        return 1.0/(1.0 + np.exp(-x))

    def forward_pass(self, x):
        self.A = {}
        self.H = {}
        self.H[0] = x.reshape(1, -1)
        for i in range(self.nh+1):
            self.A[i+1] = np.matmul(self.H[i], self.W[i+1]) + self.B[i+1]
            self.H[i+1] = self.sigmoid(self.A[i+1])
        return self.H[self.nh+1]

    def grad_sigmoid(self, x):
        return x*(1-x)

    def grad(self, x, y):
        self.forward_pass(x)
        self.dW = {}
        self.dB = {}
        self.dH = {}
```



```

self.dA = {}
L = self.nh + 1
self.dA[L] = (self.H[L] - y)
for k in range(L, 0, -1):
    self.dW[k] = np.matmul(self.H[k-1].T, self.dA[k])
    self.dB[k] = self.dA[k]
    self.dH[k-1] = np.matmul(self.dA[k], self.W[k].T)

    self.dA[k-1] = np.multiply(self.dH[k-1], self.grad_sigmoid(self.H[k-1]))

def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, display_loss=False):

    # initialise w, b
    if initialise:
        for i in range(self.nh+1):
            self.W[i+1] = np.random.randn(self.sizes[i], self.sizes[i+1])
            self.B[i+1] = np.zeros((1, self.sizes[i+1]))

    if display_loss:
        loss = {}

    for e in tqdm_notebook(range(epochs), total=epochs, unit="epoch"):
        dW = {}
        dB = {}
        for i in range(self.nh+1):
            dW[i+1] = np.zeros((self.sizes[i], self.sizes[i+1]))
            dB[i+1] = np.zeros((1, self.sizes[i+1]))
        for x, y in zip(X, Y):
            self.grad(x, y)
            for i in range(self.nh+1):
                dW[i+1] += self.dW[i+1]
                dB[i+1] += self.dB[i+1]

        m = X.shape[1]
        for i in range(self.nh+1):
            self.W[i+1] -= learning_rate * dW[i+1] / m
            self.B[i+1] -= learning_rate * dB[i+1] / m

        if display_loss:
            Y_pred = self.predict(X)
            loss[e] = mean_squared_error(Y_pred, Y)

    if display_loss:
        plt.plot(loss.values())
        plt.xlabel('Epochs')
        plt.ylabel('Mean Squared Error')
        plt.show()

def predict(self, X):

```

```

Y_pred = []
for x in X:
    y_pred = self.forward_pass(x)
    Y_pred.append(y_pred)
return np.array(Y_pred).squeeze()

```

Function by function explanation,

```

def __init__(self, n_inputs, hidden_sizes=[2]):
    #intialize the inputs
    self.nx = n_inputs
    self.ny = 1 #one final neuron for binary classification.
    self.nh = len(hidden_sizes)
    self.sizes = [self.nx] + hidden_sizes + [self.ny]
    .....

```

The `__init__` function takes a few arguments,

`n_inputs` — Number of inputs going into the network.

`hidden_sizes` — Expects a list of integers, represents the number of neurons present in the hidden layer.

- `[2]`— One hidden layer with 2 neurons
- `[2,3]`— Two hidden layers with 2 neurons in the first layer and the 3 neurons in the second layer.

In this function, we initialize two dictionaries **W** and **B** to store the randomly initialized weights and biases for each hidden layer in the network.

```
def forward_pass(self, x):
    self.A = {}
    self.H = {}
    self.H[0] = x.reshape(1, -1)
    ....
```

In the `forward_pass` function, we have initialized two dictionaries **A** and **H** and instead of representing the inputs as **X** I am representing it as H_0 so that we can save that in the post-activation dictionary **H**. Then, we will loop through all the layers and compute the pre-activation & post-activation values and store them in their respective dictionaries. The pre-activation output of the final layer is the same as the predicted value of our network. The function will return this value outside. So that we can use this value to calculate the loss of the neuron.

Remember that in the previous class *FirstFFNetwork*, we have hardcoded the computation of pre-activation and post-activation for each neuron separately but this not the case in our generic class.

```
def grad_sigmoid(self, x):
    return x*(1-x)
def grad(self, x, y):
    self.forward_pass(x)
    .....
```

Next, we define two functions which help to compute the partial derivatives of the parameters with

respect to the loss function.

```
def fit(self, X, Y, epochs=1, learning_rate=1, initialise=True, display_loss=False):
    # initialise w, b
    if initialise:
        for i in range(self.nh+1):
            self.W[i+1] = np.random.randn(self.sizes[i], self.sizes[i+1])
            self.B[i+1] = np.zeros((1, self.sizes[i+1]))
```

Then, we define our `fit` function which is essentially the same but in here we loop through each of the input and update the weights and biases in generalized fashion rather than updating the individual parameter.

```
def predict(self, X):
    #predicting the results on unseen data
    .....
```

Finally, we have the `predict` function that takes a large set of values as inputs and compute the predicted value for each input by calling the `forward_pass` function on each of the input.

Train Generic Class for Feedforward Neural Network

We will now train our data on the Generic Feedforward network which we created. First, we instantiate the `FFSNetwork` class and then call the `fit` method on the training data with 2000 epochs and learning

rate set to 0.01.

```
#train the network with two hidden layers - 2 neurons and 3 neurons

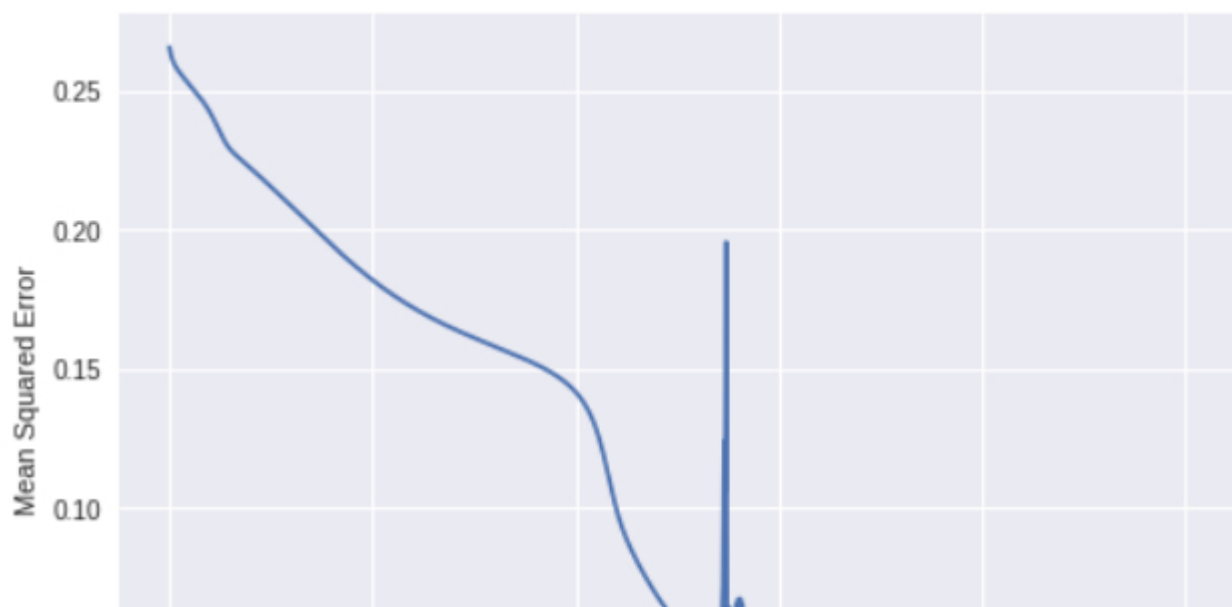
ffsnn = FFSNNetwork(2, [2, 3])
ffsnn.fit(X_train, Y_train, epochs=1000, learning_rate=.001, display_loss=True)

Y_pred_train = ffsnn.predict(X_train)
Y_pred_binarised_train = (Y_pred_train >= 0.5).astype("int").ravel()
Y_pred_val = ffsnn.predict(X_val)
Y_pred_binarised_val = (Y_pred_val >= 0.5).astype("int").ravel()
accuracy_train = accuracy_score(Y_pred_binarised_train, Y_train)
accuracy_val = accuracy_score(Y_pred_binarised_val, Y_val)

print("Training accuracy", round(accuracy_train, 2))
print("Validation accuracy", round(accuracy_val, 2))

#visualize the results
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```

The variation of loss for the neural network for training data is given below,

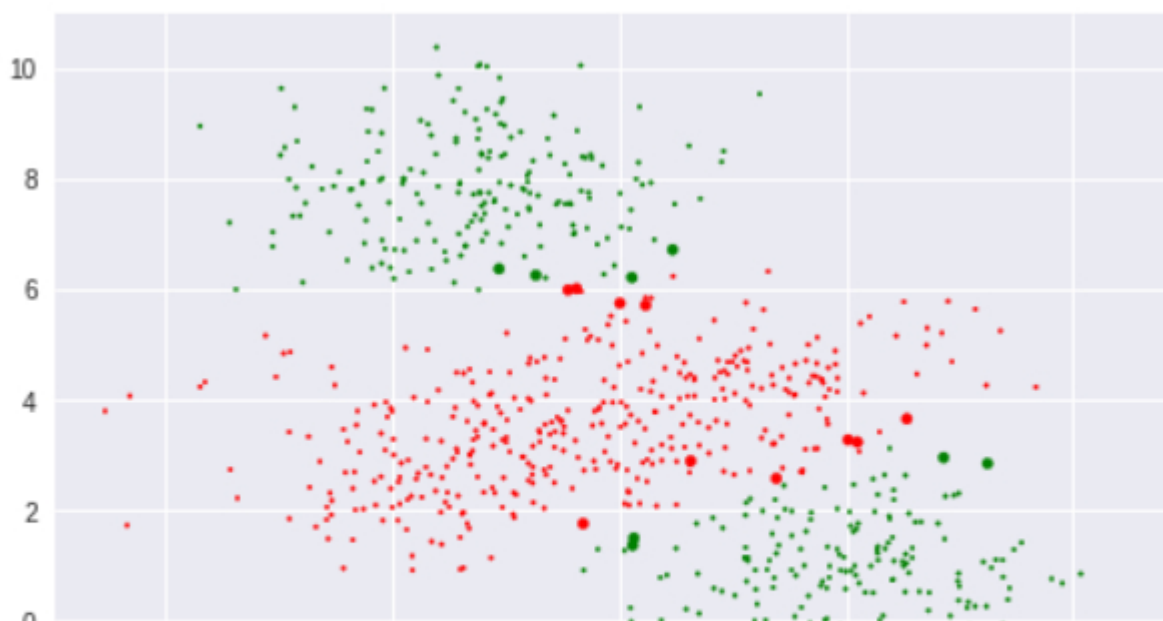


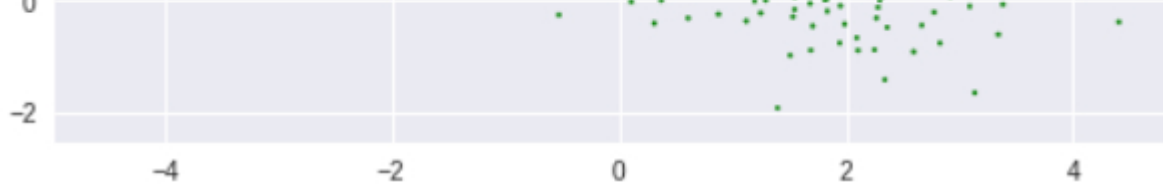


From the plot, we see that the loss function falls a bit slower than the previous network because in this case, we have two hidden layers with 2 and 3 neurons respectively. Because it is a large network with more parameters, the learning algorithm takes more time to learn all the parameters and propagate the loss through the network.

```
#visualize the predictions
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_binarised_train, cmap=my_cmap, s=15*
(np.abs(Y_pred_binarised_train-Y_train)+.2))
plt.show()
```

Again we will use the same 4D plot to visualize the predictions of our generic network. Remember that, small points indicate these observations are correctly classified and large points indicate these observations are miss-classified.





You can play with the number of epochs and the learning rate and see if can push the error lower than the current value. Also, you can create a much deeper network with many neurons in each layer and see how that network performs.

Generic FF Class for Multi-Class Classification

In this section, we will extend our generic function written in the previous section to support multi-class classification. Before we proceed to build our generic class, we need to do some data preprocessing.

Remember that initially, we generated the data with 4 classes and then we converted that multi-class data to binary class data. In this section, we will use that original data to train our multi-class neural network.

```
#remember that we have label_orig with four classes.
#split that data into train and val
X_train, X_val, Y_train, Y_val = train_test_split(data, labels_orig, stratify=labels_orig,
random_state=0)
print(X_train.shape, X_val.shape, labels_orig.shape)

#one hot encoder
enc = OneHotEncoder()
# 0 -> (1, 0, 0, 0), 1 -> (0, 1, 0, 0), 2 -> (0, 0, 1, 0), 3 -> (0, 0, 0, 1)
y_OH_train = enc.fit_transform(np.expand_dims(Y_train,1)).toarray()
y_OH_val = enc.fit_transform(np.expand_dims(Y_val,1)).toarray()
print(y_OH_train.shape, y_OH_val.shape)
```

Here we have 4 different classes, so we encode each label so that the machine can understand and do computations on top of it. To encode the labels, we will use `sklearn.OneHotEncoder` on training and validation labels.

We will write our generic feedforward network for multi-class classification in a class called `FFSN_MultiClass`.

```
class FFSN_MultiClass:

    def __init__(self, n_inputs, n_outputs, hidden_sizes=[3]):
        self.nx = n_inputs
        self.ny = n_outputs
        self.nh = len(hidden_sizes)
        self.sizes = [self.nx] + hidden_sizes + [self.ny]

        self.W = {}
        self.B = {}
        for i in range(self.nh+1):
            self.W[i+1] = np.random.randn(self.sizes[i], self.sizes[i+1])
            self.B[i+1] = np.zeros((1, self.sizes[i+1]))

    def sigmoid(self, x):
        return 1.0/(1.0 + np.exp(-x))

    def softmax(self, x):
        exps = np.exp(x)
        return exps / np.sum(exps)

    def forward_pass(self, x):
        self.A = {}
        self.H = {}
        self.H[0] = x.reshape(1, -1)
        for i in range(self.nh):
            self.A[i+1] = np.matmul(self.H[i], self.W[i+1]) + self.B[i+1]
            self.H[i+1] = self.sigmoid(self.A[i+1])
        self.A[self.nh+1] = np.matmul(self.H[self.nh], self.W[self.nh+1]) + self.B[self.nh+1]
        self.H[self.nh+1] = self.softmax(self.A[self.nh+1])
        return self.H[self.nh+1]
```



```

def predict(self, X):
    Y_pred = []
    for x in X:
        y_pred = self.forward_pass(x)
        Y_pred.append(y_pred)
    return np.array(Y_pred).squeeze()

def grad_sigmoid(self, x):
    return x*(1-x)

def cross_entropy(self, label, pred):
    y1=np.multiply(pred,label)
    y1=y1[y1!=0]
    y1=-np.log(y1)
    y1=np.mean(y1)
    return y1

def grad(self, x, y):
    self.forward_pass(x)
    self.dW = {}
    self.dB = {}
    self.dH = {}
    self.dA = {}
    L = self.nh + 1
    self.dA[L] = (self.H[L] - y)
    for k in range(L, 0, -1):
        self.dW[k] = np.matmul(self.H[k-1].T, self.dA[k])
        self.dB[k] = self.dA[k]
        self.dH[k-1] = np.matmul(self.dA[k], self.W[k].T)
        self.dA[k-1] = np.multiply(self.dH[k-1], self.grad_sigmoid(self.H[k-1]))

def fit(self, X, Y, epochs=100, initialize='True', learning_rate=0.01,
display_loss=False):

    if display_loss:
        loss = {}

    if initialize:
        for i in range(self.nh+1):
            self.W[i+1] = np.random.randn(self.sizes[i], self.sizes[i+1])
            self.B[i+1] = np.zeros((1, self.sizes[i+1]))

    for epoch in tqdm_notebook(range(epochs), total=epochs, unit="epoch"):
        dW = {}
        dB = {}
        for i in range(self.nh+1):
            dW[i+1] = np.zeros((self.sizes[i], self.sizes[i+1]))

```

```

        dB[i+1] = np.zeros((1, self.sizes[i+1]))
    for x, y in zip(X, Y):
        self.grad(x, y)
        for i in range(self.nh+1):
            dW[i+1] += self.dW[i+1]
            dB[i+1] += self.dB[i+1]

    m = X.shape[1]
    for i in range(self.nh+1):
        self.W[i+1] -= learning_rate * (dW[i+1]/m)

        self.B[i+1] -= learning_rate * (dB[i+1]/m)

    if display_loss:
        Y_pred = self.predict(X)
        loss[epoch] = self.cross_entropy(Y, Y_pred)

if display_loss:
    plt.plot(loss.values())
    plt.xlabel('Epochs')
    plt.ylabel('CE')
    plt.show()

```

I will explain changes what are the changes made in our previous class FFSNetwork to make it work for multi-class classification.

First, we have our `forward_pass` function,

```

def forward_pass(self, x):
    self.A = {}
    self.H = {}
    self.H[0] = x.reshape(1, -1)
    for i in range(self.nh):
        self.A[i+1] = np.matmul(self.H[i], self.W[i+1]) + self.B[i+1]
        self.H[i+1] = self.sigmoid(self.A[i+1])
    self.A[self.nh+1] = np.matmul(self.H[self.nh], self.W[self.nh+1]) + self.B[self.nh+1]
    self.H[self.nh+1] = self.softmax(self.A[self.nh+1])

```

```
return self.H[self.nh+1]
```

Since we have multi-class output from the network, we are using softmax activation instead of sigmoid activation at the output layer. At *Line 29–30* we are using softmax layer to compute the forward pass at the output layer.

```
def cross_entropy(self, label, pred):  
    y1=np.multiply(pred,label)  
    y1=y1[y1!=0]  
    y1=-np.log(y1)  
    y1=np.mean(y1)  
    return y1
```

Next, we have our loss function. In this case, instead of the mean square error, we are using the cross-entropy loss function. By using the cross-entropy loss we can find the difference between the predicted probability distribution and actual probability distribution to compute the loss of the network.

Train Generic Class for Multi-Class Classification

We will now train our data on the Generic Multi-Class Feedforward network which we created. First, we instantiate the FFSN_MultiClass Class and then call the fit method on the training data with 2000 epochs and learning rate set to 0.005. Remember that our data has two inputs and 4 encoded labels.

```
#train the network
```

```
ffsn_multi = FFSN_MultiClass(2,4,[2,3])
ffsn_multi.fit(X_train,y_OH_train,epochs=2000,learning_rate=.005,display_loss=True)

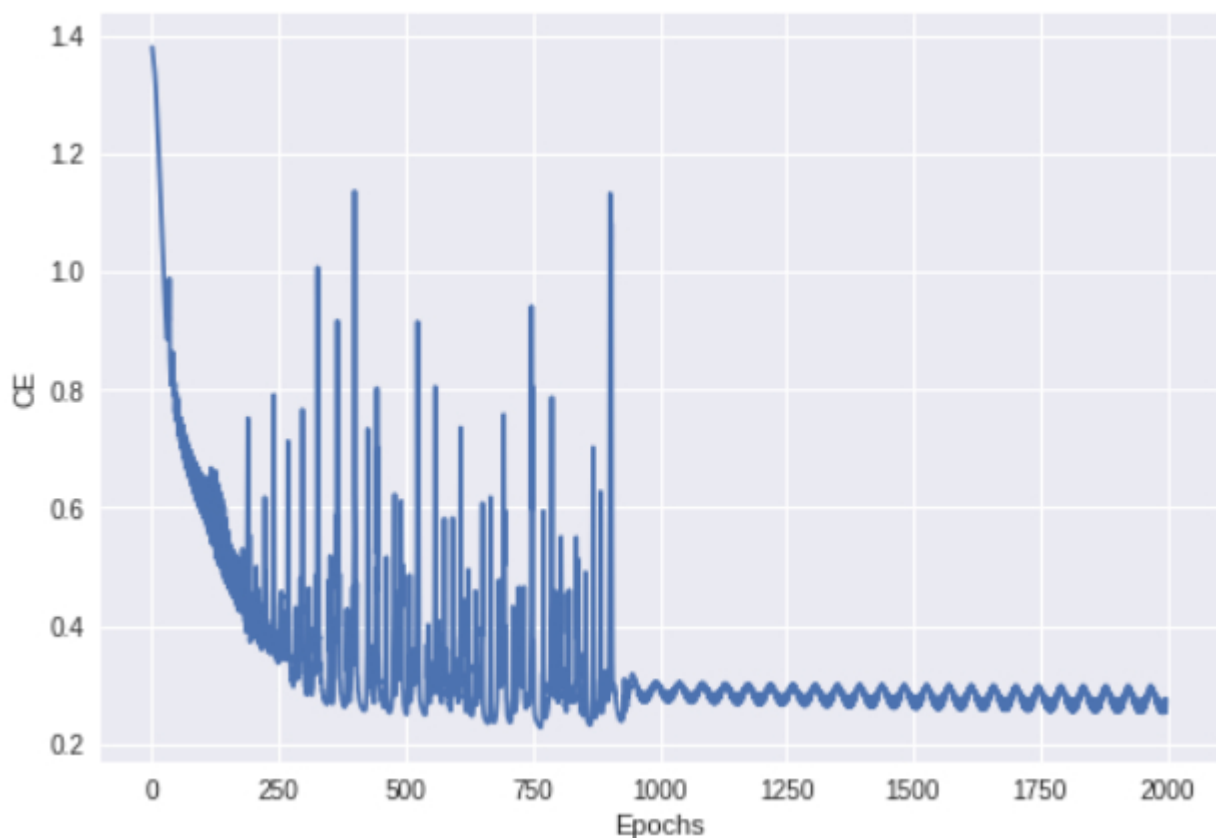
Y_pred_train = ffsn_multi.predict(X_train)
Y_pred_train = np.argmax(Y_pred_train,1)

Y_pred_val = ffsn_multi.predict(X_val)
Y_pred_val = np.argmax(Y_pred_val,1)

accuracy_train = accuracy_score(Y_pred_train, Y_train)
accuracy_val = accuracy_score(Y_pred_val, Y_val)

print("Training accuracy", round(accuracy_train, 2))
print("Validation accuracy", round(accuracy_val, 2))
#visualize
plt.scatter(X_train[:,0], X_train[:,1], c=Y_pred_train, cmap=my_cmap, s=15*
(np.abs(np.sign(Y_pred_train-Y_train))+.1))
plt.show()
```

The variation of loss for the neural network for training data is given below,



Again we will use the same 4D plot to visualize the predictions of our generic network. To plot the

graph we need to get the one final predicted label from the network, in order to get that predicted value I have applied the `argmax` function to get the label with the highest probability. Using that label we can plot our 4D graph and compare it with the actual input data scatter plot.



There you have it, we have successfully built our generic neural network for multi-class classification from scratch.



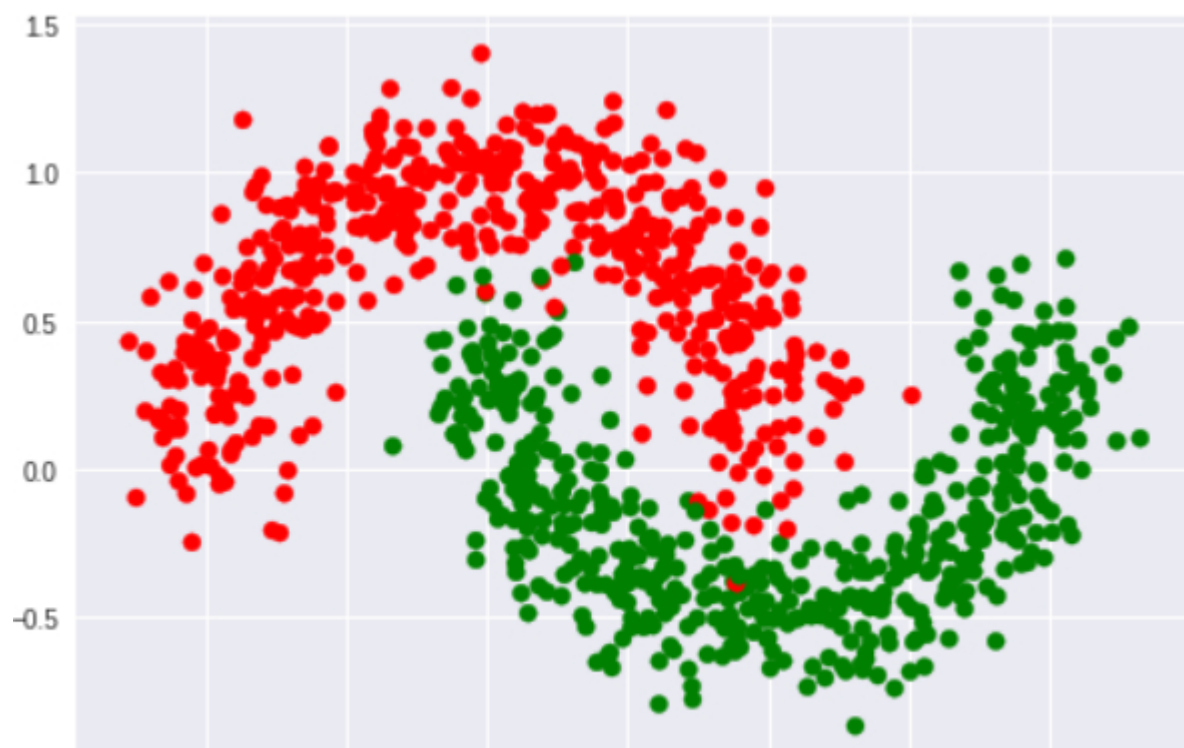


Photo by Vasily Koloda on Unsplash

What's Next?

LEARN BY CODING

In this article, we have used `make_blobs` function to generate toy data and we have seen that `make_blobs` generate linearly separable data. If you want to generate some complex non-linearly separable data to train your feedforward neural network, you can use `make_moons` function from `sklearn` package.



Make Moons Function Data

The `make_moons` function generates two interleaving half circular data essentially gives you a non-linearly separable data. Also, you can add some Gaussian noise into the data to make it more complex for the neural network to arrive at a non-linearly separable decision boundary.

Using our generic neural network class you can create a much deeper network with more number of neurons in each layer (also different number of neurons in each layer) and play with learning rate & a number of epochs to check under which parameters neural network is able to arrive at best decision boundary possible.

The entire code discussed in the article is present in this [GitHub repository](#). Feel free to fork it or download it.

Niranjankumar-c/Feedforward_NeuralNetwork

Conclusion

In this post, we have built a simple neuron network from scratch and seen that it performs well while our sigmoid neuron couldn't handle non-linearly separable data. Then we have seen how to write a generic class which can take '`n`' number of inputs and '`L`' number of hidden layers (with many neurons for each layer) for binary classification using mean squared error as loss function. After that, we extended our generic class to handle multi-class classification using softmax and cross-entropy as loss function and saw that it's performing reasonably well.

Continue Learning

if you are interested in learning more about Artificial Neural Network, check out the Artificial Neural Networks by Abhishek and Pukhraj from Starttechacademy. Also, this course will be taught in the latest version of Tensorflow 2.0 (Keras backend).

They also have a very good bundle on machine learning (Basics + Advanced) in both Python and R languages.

Recommended Reading

Deep Learning: Feedforward Neural Networks Explained

In my next post, I will explain backpropagation in detail along with some math. So make sure you follow me on medium to get notified as soon as it drops.

Until then Peace :)

NK.

Disclaimer — There might be some affiliate links in this post to relevant resources. You can purchase

the bundle at the lowest price possible. I will receive a small commission if you purchase the course.

6    

Share this story



@NKumar

Niranjankumar

😊 Data Science Courses Holiday Sale

Related Stories

[Fill HackerEarth Developer Survey 2021 & win Amazon gift vouchers by @HackerEarth](#)

[Getting Started With Pytorch In Google Collab With Free GPU by @NKumar](#)

[What are Insecure Direct Object References \(IDOR\)? by @hacker0x01](#)

[Can bitcoin-denominated UBI fight hyperinflation? by @ethan88](#)

[What Is Synthetic Data And How It Works by @innotescus](#)

[The #Decentralized-Internet Writing Contest by @David](#)

[What the heck is EIP-1559? by @haroonbaig](#)

Subject Matter

[#promoted](#)

[#artificial-intelligence](#)

[#good-company](#)

[#bitcoin](#)

[#data-labelling](#)

[#decentralized-internet](#)

[#cryptocurrency](#)

TAGS

[#feedforward-neural](#)

[#artificial-neural-network](#)

[#machine-learning](#)

[#hackernoon-top-story](#)

[#deep-neural-networks](#)

